

WS 2025/26

HUMAN ROBOT INTERACTION LAB COURSE

FINAL COURSE REPORT

Chair of Autonomous Systems and Mechatronics
Friedrich-Alexander-Universität Erlangen-Nürnberg

TEAM 1 :

Velayudhan Aravindakshan - ju08rufe
Akhilan David Karthikeyan - ug39owys
Wei Zhang - jo07wuwe

Task 1: Robot Manipulation with ROS

- **a) Planning Considerations:** To ensure execution is possible, one must consider joint limits to avoid mechanical failure, velocity/acceleration limits of motors, the execution rate in `rospy.rate()` to prevent controller overload, and collision avoidance for the robot's physical links.
- **b) End Effector Behaviour:** When traversing a trajectory planned linearly in joint space, the end effector moves in a non-linear, curved arc because the kinematic mapping (trigonometric functions) from joint to Cartesian space is non-linear.
- **c) Command Levels:** Velocity (or rate) control is independent of the starting position, as it commands a rate of change rather than an absolute target.
- **d) Linear Workspace Motion:** To reach a specific point or move linearly, one uses Inverse Kinematics (IK) to solve for required joint angles and Cartesian Trajectory Planning to define intermediate waypoints for the IK solver to follow.
- **e) Orientation Calculation:** Orientation can be calculated from a rotation matrix using Euler Angles or Roll-Pitch-Yaw (RPY) sequences (e.g., Z-Y-X). This involves equating the calculated matrix to a mathematical expansion of the angles and solving the resulting trigonometric equations.

Task 2: Kinematics and Interaction

- **a) IK Problems & Comparison:** Converting Cartesian trajectories to joint space can cause discontinuities because IK solutions for redundant robots are non-unique.
 - Cartesian Planning: Predictable straight paths, but computationally complex and prone to singularities.
 - Joint Planning: Simpler and guaranteed continuous motion, but results in unpredictable curved paths.
- **b) Event Transmission:** ROS Services (synchronous request/reply) or ROS Actions (asynchronous, long-running tasks with feedback) are more suitable for distinct events than constant streams.

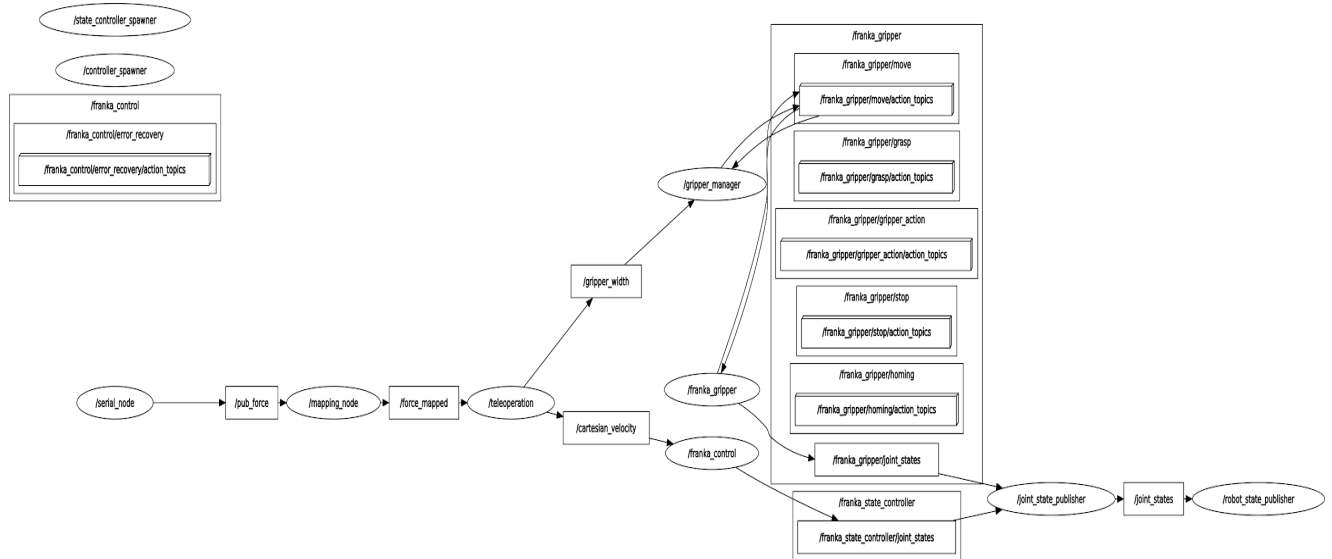
- **c) Force Measurement:** Deriving forces from joint moments is difficult because measurements include gravity, inertia, and friction, requiring an exact dynamic model for isolation.
 - End Effector Sensor: Precise, but adds bulk and only measures tip forces.
 - Joint Torque Sensor: Covers the whole structure but requires complex modelling.
 - Limitation: Forces aligned perfectly with a joint axis cannot be measured by that joint sensor.
- **d) Link Interaction:** Under task space impedance control, interacting with links causes the robot to react compliantly due to measured joint moments, but the resulting end effector motion may not align perfectly with the contact point's yielding as the loop optimises for the tip.

Task 3: Cooperative Lifting

- **a) Paper Review & Lab Experience:** The Marullo paper found the Extended Patch (EP) method superior to Single Point (SP) in completion time and rotational stability. Our lab experience mirrored the SP failure: slight misalignments led to instability and jerky behaviour.
- **b) Control Modalities:**
 - Position Level: Used in Tasks 1 & 2 for trajectories.
 - Velocity Level: Used in Task 3 for cooperative lifting.
 - Torque Level: Internally used in Task 2 for impedance control.
 - Others: Acceleration Control and Admittance Control.
- **c) ROS Services vs. Actions:** Services are synchronous and for short tasks; Actions are asynchronous, provide feedback, and are preemptable for long-running tasks.

Task 4: Teleoperation Pipeline

Teleoperation Pipeline Flowchart - [click here for the flowchart clear view](#)(please note due to word document resolution and automatic image compression the flow chart may visually be distorted)



- **Pipeline Logic:** The `/serial_node` provides raw force data to the `/mapping_node`, which passes normalised values to the `/teleoperation` node. This node processes gamepad input to publish `/cartesian_velocity` and `/gripper_width`. These are executed by `/franka_control` and `/gripper_manager`, respectively.

Task 5: Verbal Interaction

- **a) Theory vs. Reality:** QiChat subrules align with the hierarchical dialogue management in the HRI textbook. However, real-world noise required adjusting confidence thresholds, highlighting embodiment challenges not present in simulation.
- **b) Multimodal Interaction:** Interactions without non-verbal elements felt mechanical; it was unclear if the robot was "listening". Adding animation tags (e.g., "hello") made the robot feel intentional and social.

Task 6: Object Recognition

- **Accuracy:** YOLOv5 (correctly detecting 4/5 objects) significantly outperformed NAO's integrated function (1-2/5 objects).
- **Learning:** NAO requires manual real-time "learning," while YOLO uses transfer learning on large datasets.

- **Algorithm:** NAO uses SIFT (Scale-Invariant Feature Transform), which relies on visual keypoints and fails on plain/reflective objects. YOLOv5 (CNN) is superior as it evaluates the entire image context.

Reflection: Our key insight from this course is that while theoretical robotic control is precise, successful human-robot interaction in the real world requires balancing complex kinematic planning with intuitive, multimodal feedback to bridge the gap between simulation and embodiment.